



ÉCOLE POLYTECHNIQUE DE THIÈS

Lab Guide: ArduPilot SITL and Gazebo Simulation for Quadcopter Flight Control and Guidance

Author: Abdourahmane Fall & Imane Attouzan

Supervisor: Dr. Birame SY & M. Mansour DIENE

Final-Year Project

Department of Aeronautical Engineering

October 13, 2025

Contents

1	Introduction	2
2	Tools Overview	2
2.1	ArduPilot SITL (Software-In-The-Loop)	2
2.2	MAVProxy	2
2.3	Gazebo	2
3	Installation Guide	2
3.1	Installing ArduPilot	3
3.2	Installing Gazebo and Plugin	3
4	Simulation Execution	3
4.1	Launching Gazebo	3
4.2	Launching ArduPilot SITL with MAVProxy	3
4.3	Autonomous Takeoff	4
4.4	Basic Navigation (Point A to Point B)	4
4.5	GPS Signal Loss Simulation	4
4.6	Waypoint Navigation	4
4.7	Automatic Return to Launch (RTL)	5
4.8	Post-Flight Analysis with MAVExplorer	5

1 Introduction

This laboratory guide describes how to set up and use ArduPilot’s Software-in-the-Loop (SITL) environment together with the Gazebo 3D simulator to understand and practice the key phases of an autonomous drone mission—from takeoff to landing. The objectives for the exercises include:

- Performing autonomous takeoff and landing sequences;
- Navigating to given positions and executing waypoint missions;
- Simulating GPS loss and studying its effect on stability and failsafes;
- Creating waypoint plans into the simulator and running them autonomously;
- Commanding a Return-To-Launch (RTL) and analyzing mission logs (using MAVExplorer).

2 Tools Overview

2.1 ArduPilot SITL (Software-In-The-Loop)

ArduPilot SITL compiles the ArduPilot code into a native executable that emulates flight dynamics and onboard sensors (GPS, accelerometers, gyros, etc.). It allows developers to run virtual multicopters, airplanes, or rovers directly on a PC—without any physical hardware. This environment is ideal for safely testing new features, debugging autopilot algorithms, and developing control logic using standard development tools.

2.2 MAVProxy

MAVProxy is a lightweight, command-line Ground Control Station (GCS) that communicates with ArduPilot through the MAVLink protocol. It displays real-time telemetry messages, provides a built-in map interface, and supports numerous commands to arm, guide, or control the drone.

2.3 Gazebo

Gazebo is a powerful open-source 3D robotics simulator that provides realistic physics and sensor simulation. It is widely used in robotics. Through the ArduPilot–Gazebo plugin, the virtual drone model (e.g., the Iris quadcopter) can be visualized in a realistic 3D world. The simulator accurately models environmental effects such as wind, terrain, and obstacles.

3 Installation Guide

This guide assumes a Linux-based environment (Ubuntu recommended). If you are using Windows or macOS, you can run Ubuntu through a virtual machine such as VMware, which provides excellent performance.

3.1 Installing ArduPilot

Step 1: Clone the ArduPilot repository

```
cd ~
sudo apt install git
git clone https://github.com/ArduPilot/ardupilot.git
```

Step 2: Install dependencies

```
cd ardupilot
Tools/environment_install/install-prereqs-ubuntu.sh -y
. ~/.profile
```

Step 3: Checkout latest Copter build

```
git checkout Copter-4.6.0
git submodule update --init --recursive
```

Step 4: If "submodule update" fails

```
git config --global url.https://.insteadOf git://
# Then repeat step 3
```

3.2 Installing Gazebo and Plugin

To install Gazebo and the ArduPilot–Gazebo plugin, follow the official ArduPilot documentation at: <https://ardupilot.org/dev/docs/sitl-with-gazebo.html>

4 Simulation Execution

4.1 Launching Gazebo

Step 5: Open a terminal and run:

```
gz sim -v4 -r iris_runway.sdf
```

If the rendering is unstable, append the following:

```
--render-engine ogre
```

Expected result: The Gazebo window opens displaying the Iris quadcopter on a runway. To lock the camera on the drone, right-click the model and select *Follow*.

4.2 Launching ArduPilot SITL with MAVProxy

Step 6: In a new terminal:

```
cd ~/ardupilot/ArduCopter
sim_vehicle.py -v ArduCopter -f gazebo-iris --model JSON --map --console
```

Expected result: MAVProxy opens with two windows: a console (telemetry output) and a live map. The autopilot connects to Gazebo through the plugin and the drone's behavior is simulated with real-time feedback.

4.3 Autonomous Takeoff

Step 7: In the MAVProxy console:

```
mode guided
arm throttle
takeoff 10
```

Expected result: The drone arms and climbs automatically to an altitude of 10 meters. The map and Gazebo display a stable ascent.

4.4 Basic Navigation (Point A to Point B)

Step 8: After takeoff, command the drone to move toward target GPS coordinates:

```
guided 47.397742 8.545594 30
```

Expected result: The drone navigates to the target position.

4.5 GPS Signal Loss Simulation

Step 9: Disable the GPS:

```
param set SIM_GPS_DISABLE 1
```

Expected result: The autopilot loses GPS input and triggers a failsafe LAND mode. The drone descends automatically.

Step 10: Re-enable the GPS:

```
param set SIM_GPS_DISABLE 0
```

Step 11: If EKF3 returns to green but the drone is still landing, switch back to GUIDED mode:

```
mode guided
```

Expected result: The drone stabilizes and maintains altitude.

4.6 Waypoint Navigation

Step 12: Define multiple waypoints using MAVProxy's mission interface:

1. Right-click on the map → **Mission** → **Draw**.
2. Add waypoints by left-clicking.
3. Right-click again → **Mission** → **Loop**.

Step 13: Execute the mission:

```
mode auto
```

Expected result: The drone switches to AUTO mode and flies sequentially through all waypoints.

4.7 Automatic Return to Launch (RTL)

Step 14: Trigger automatic return:

```
mode rtl
```

Expected result: The drone flies back to home, and lands automatically.

4.8 Post-Flight Analysis with MAVExplorer

Step 15: After the flight, stop MAVProxy automatically generates flight logs (`mav.tlog`, `mav.tlog.raw`) in the working directory.

Step 16: Open the log with MAVExplorer:

```
MAVExplorer.py mav.tlog
```

Expected result: MAVExplorer opens with a graphical interface. The **Graphs** tab automatically displays several curves (speed, attitude, battery levels, etc.) based on the available data. The **Display->Map** tab shows the GPS flight path on a map with the different modes active.